Базы данных. SQL

Обо мне

Выпускник школы программистов hh
2021-2022 гг.

Бэкенд-разработчик в команде A-Shot

План

Что такое БД и немного теории

Нормальные формы

SQL

Транзакции

Индексы

Зачем нужны базы данных

Зачем нужны базы данных

…и какие они бывают

Зачем?

1. Вокруг нас много разных данных
2. Данные нужно как-то хранить

Зачем нужны базы данных

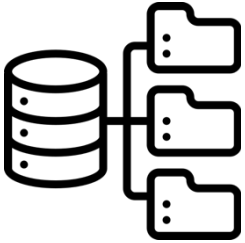
База данных и СУБД

Зачем нужны базы данных

База данных и СУБД

База данных

Набор данных, организованных каким-то способом.

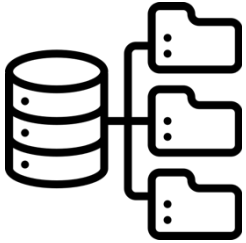


Зачем нужны базы данных

База данных и СУБД

База данных

Набор данных, организованных каким-то способом.



СУБД

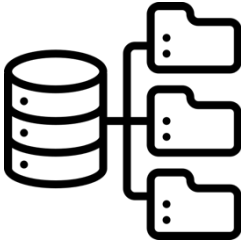
Программа для манипуляции данными в базе

Зачем нужны базы данных

База данных и СУБД

База данных

Набор данных, организованных каким-то способом.



≠

СУБД

Программа для манипуляции данными в базе

Microsoft SQL Server ORACLE IBM DB2 PostgreSQL

Зачем нужны базы данных

Для чего нужны?

Зачем нужны базы данных

Основные задачи БД

1. Сохранить данные по запросу

Зачем нужны базы данных

Основные задачи БД

1. Сохранить данные по запросу
2. Изменить данные по запросу

Зачем нужны базы данных

Основные задачи БД

1. Сохранить данные по запросу
2. Изменить данные по запросу
3. Найти данные по запросу

Зачем нужны базы данных

Основные задачи БД

1. Сохранить данные по запросу
2. Изменить данные по запросу
3. Найти данные по запросу
4. Не дать прочитать эти данные тем, кому не следует, а кому надо — дать

Зачем нужны базы данных

Основные задачи БД

1. Сохранить данные по запросу
2. Изменить данные по запросу
3. Найти данные по запросу
4. Не дать прочитать эти данные тем, кому не следует, а кому надо — дать
5. Поддерживать порядок

Зачем нужны базы данных

Основные задачи БД

1. Сохранить данные по запросу
2. Изменить данные по запросу
3. Найти данные по запросу
4. Не дать прочитать эти данные тем, кому не следует, а кому надо — дать
5. Поддерживать порядок
6. Масштабироваться

Зачем нужны базы данных

Основные задачи БД

1. Сохранить данные по запросу
2. Изменить данные по запросу
3. Найти данные по запросу
4. Не дать прочитать эти данные тем, кому не следует, а кому надо — дать
5. Поддерживать порядок
6. Масштабироваться
7. Не терять данные

Преимущества БД

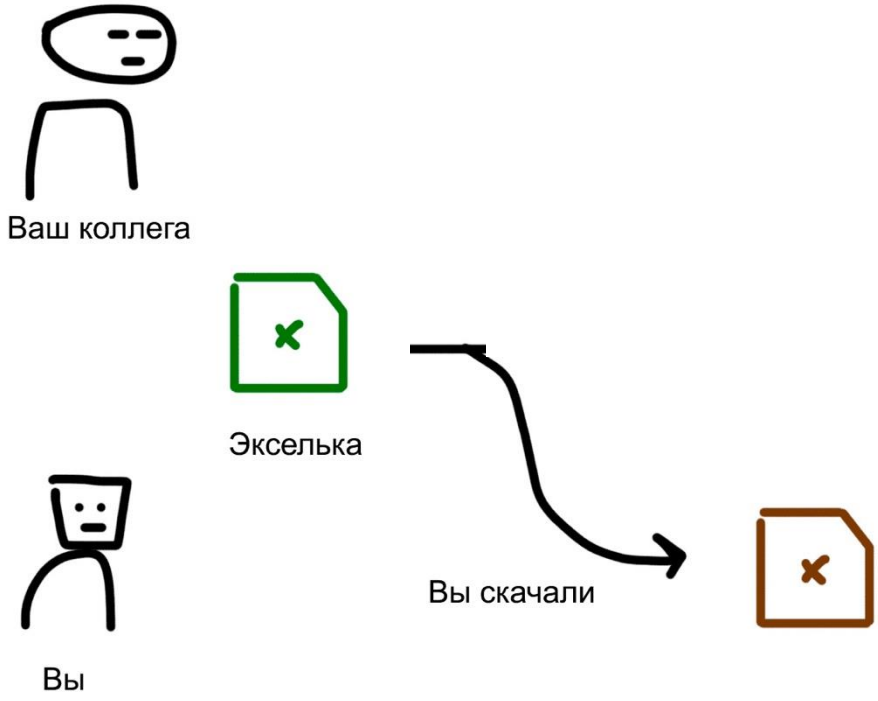
Преимущества БД

Когда нет СУБД и нужно поработать с данными

Файл в сетевом хранилище

Преимущества БД

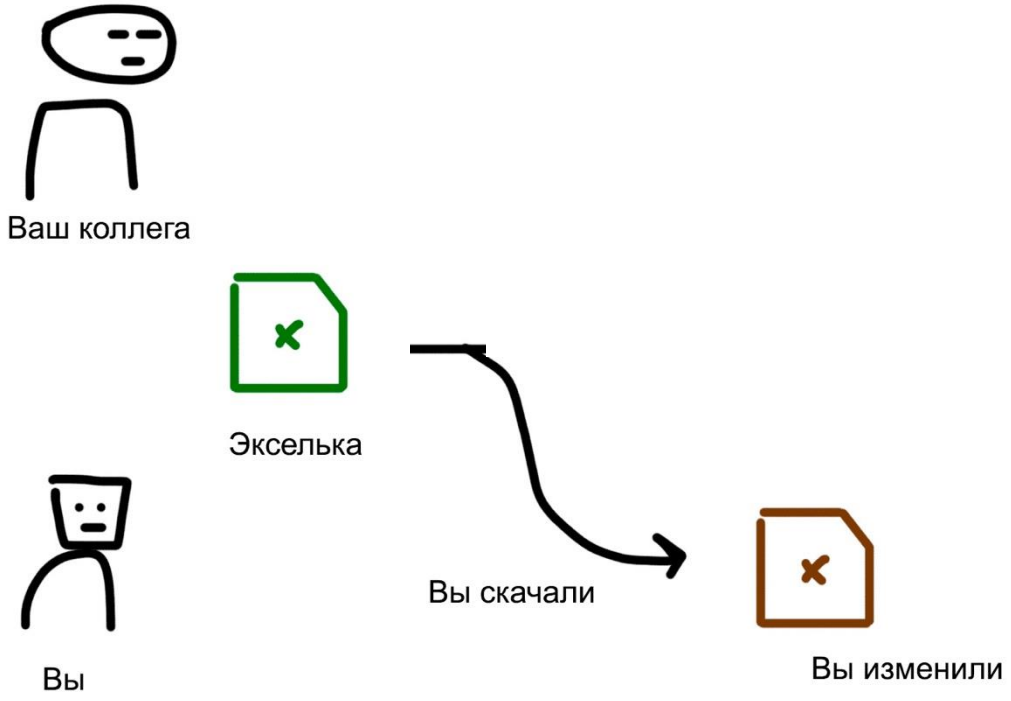
Когда нет СУБД и нужно поработать с данными



Файл в сетевом хранилище

Преимущества БД

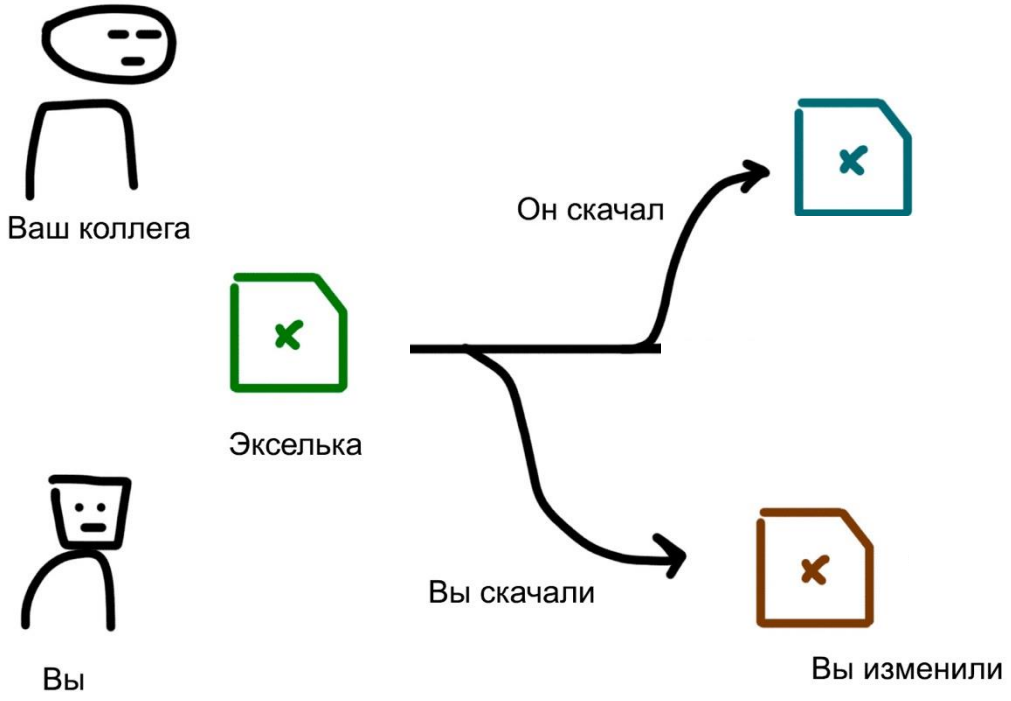
Когда нет СУБД и нужно поработать с данными



Файл в сетевом хранилище

Преимущества БД

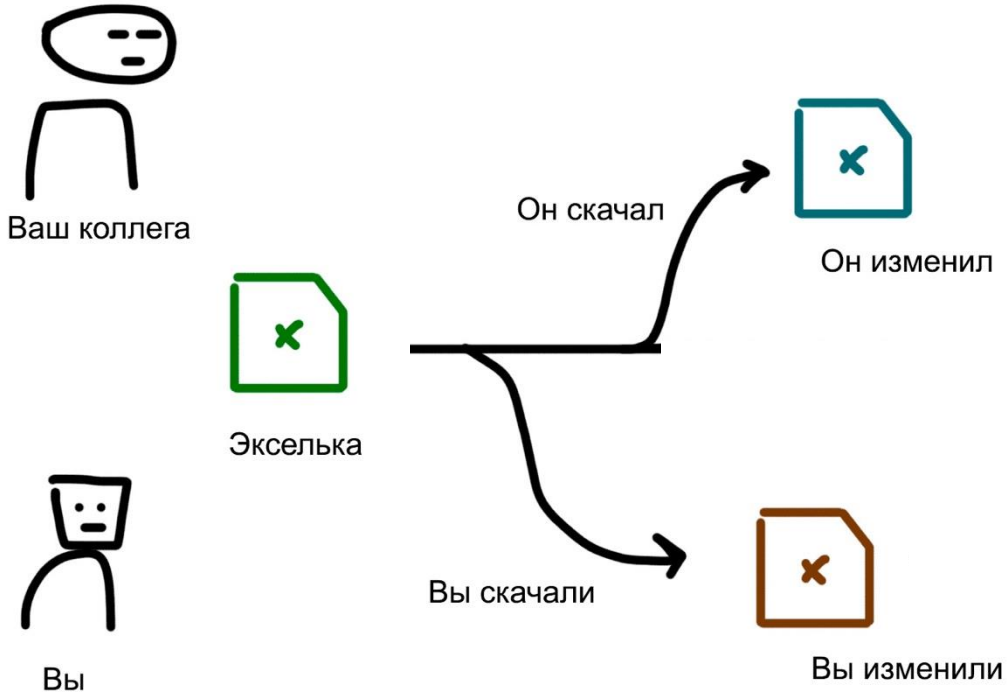
Когда нет СУБД и нужно поработать с данными



Файл в сетевом хранилище

Преимущества БД

Когда нет СУБД и нужно поработать с данными

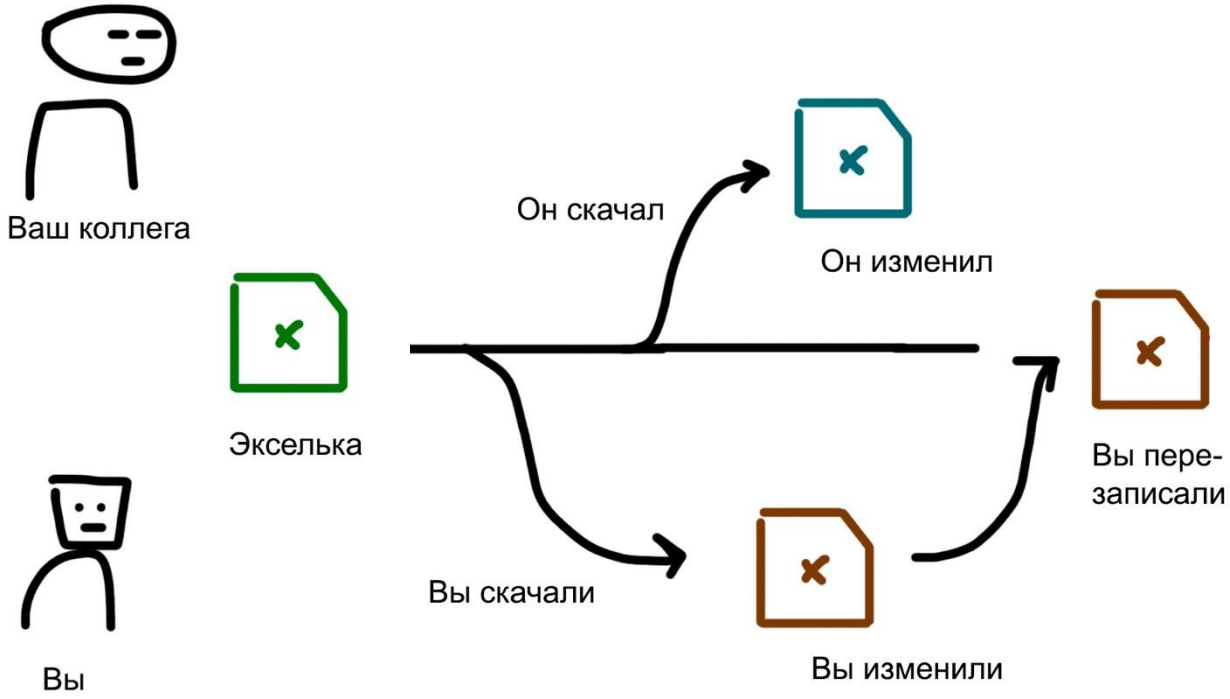


Файл в сетевом хранилище

Преимущества БД

Когда нет СУБД и нужно поработать с данными

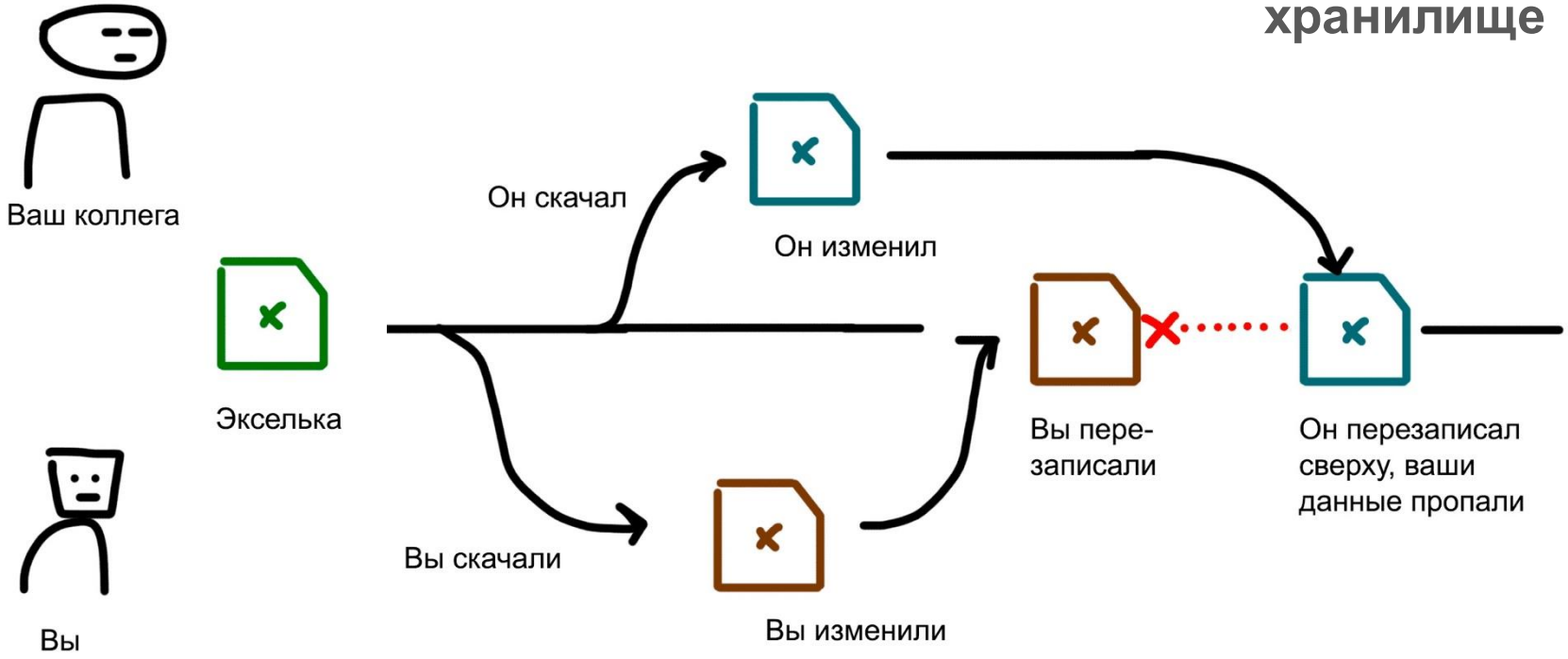
Файл в сетевом хранилище



Преимущества БД

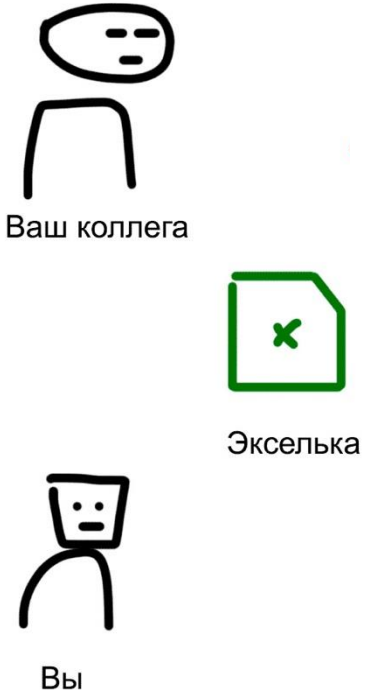
Когда нет СУБД и нужно поработать с данными

Файл в сетевом хранилище



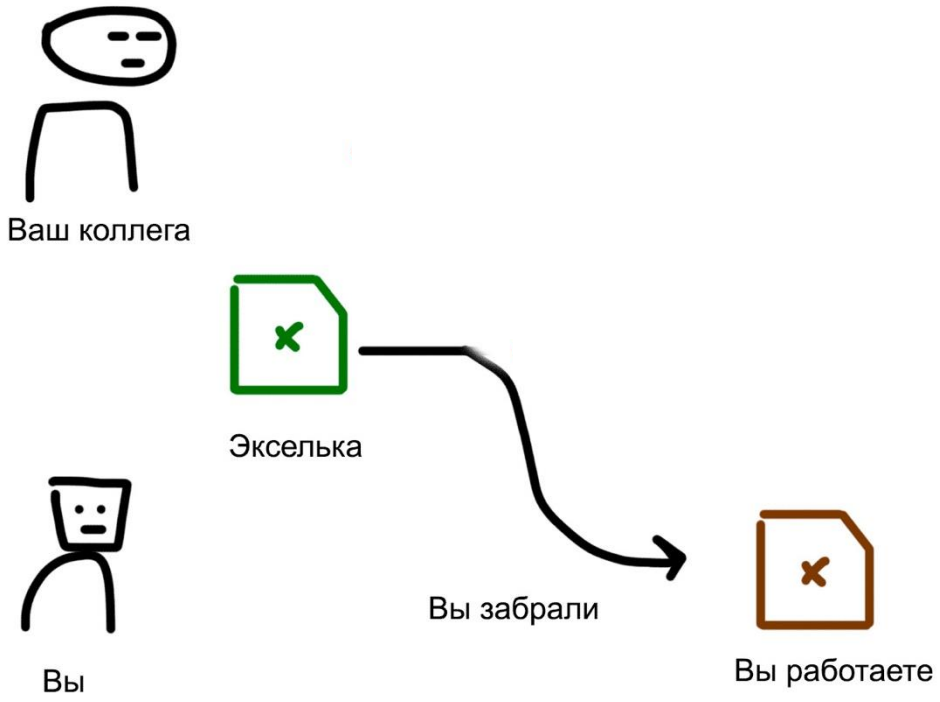
Преимущества БД

Файл на флешке



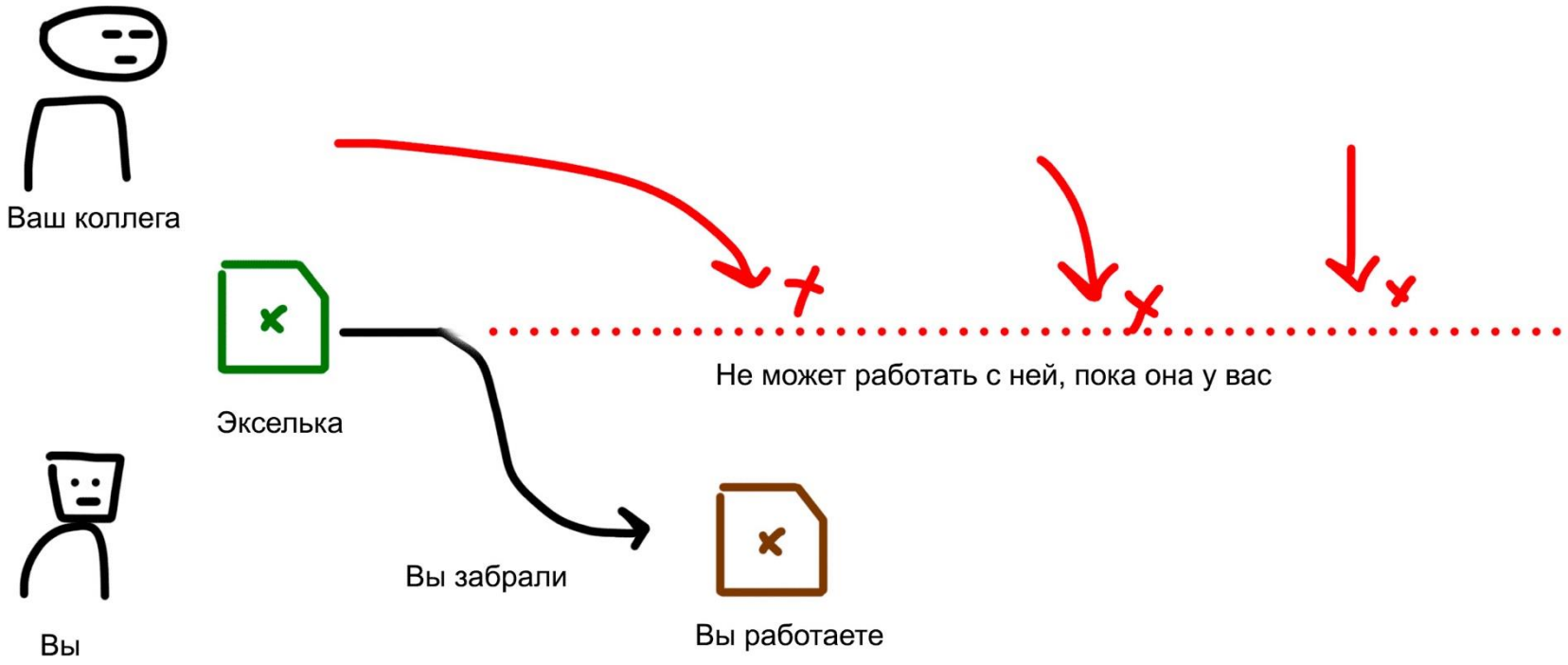
Преимущества БД

Файл на флешке



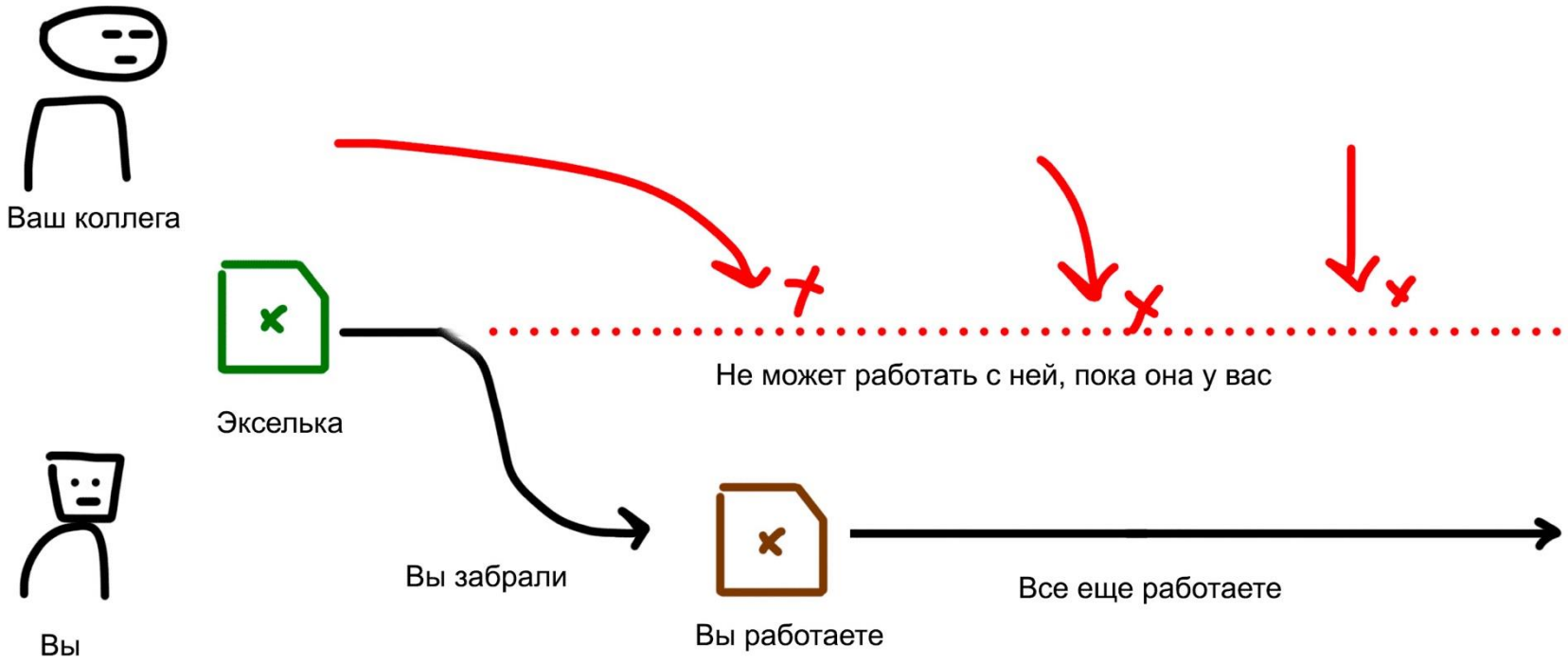
Преимущества БД

Файл на флешке



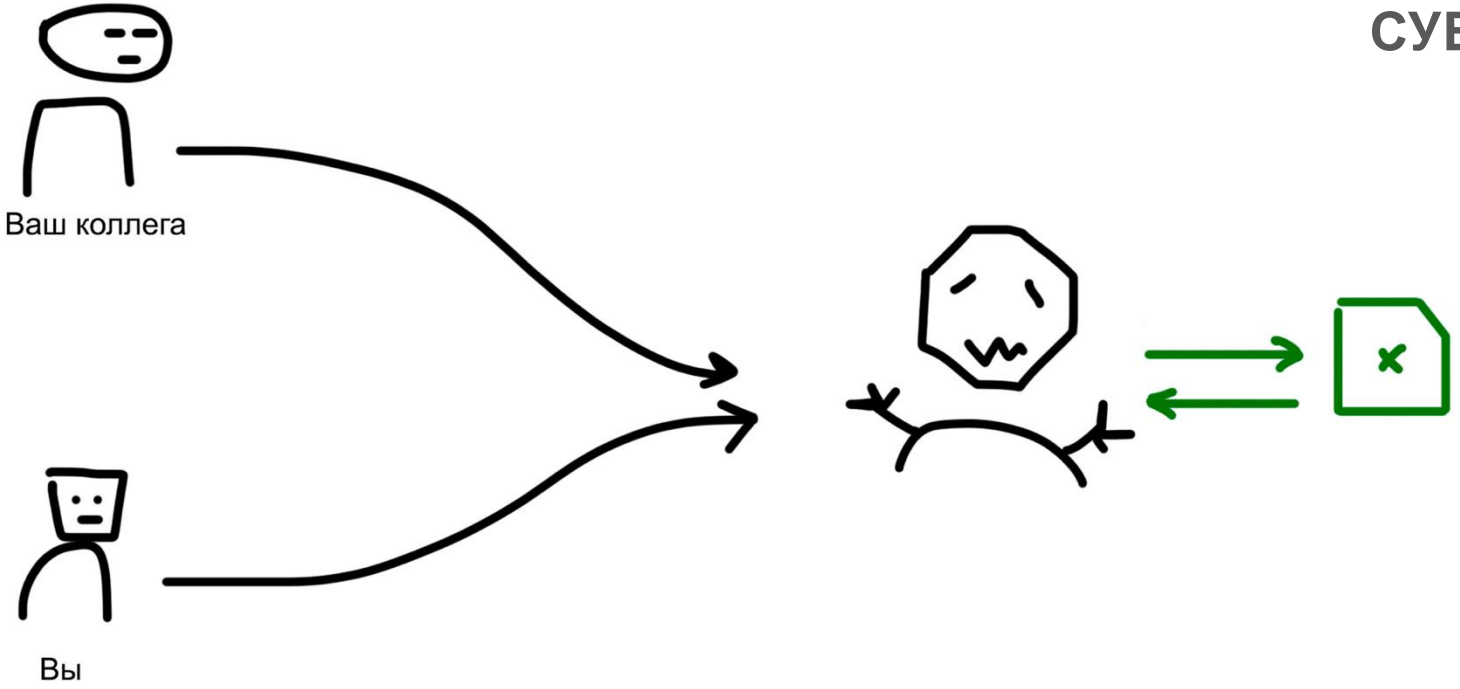
Преимущества БД

Файл на флешке



Преимущества БД

Сделать свою СУБД



ВОПРОСЫ

Типы баз данных

Типы баз данных

1. Реляционные

Типы баз данных

1. Реляционные
2. Документоориентированные

Типы баз данных

1. Реляционные
2. Документоориентированные
3. Key-value

Типы баз данных

1. Реляционные
2. Документоориентированные
3. Key-value
4. Колоночные

Типы баз данных

1. Реляционные
2. Документоориентированные
3. Key-value
4. Колоночные
5. Иерархические

Типы баз данных

1. Реляционные
2. Документоориентированные
3. Key-value
4. Колоночные
5. Иерархические
6. Сетевые
7. …

Типы баз данных

1. Реляционные
2. Документоориентированные
3. Key-value
4. Колоночные
5. Иерархические
6. Сетевые
7. …

Реляционные БД

Реляционные БД

Реляционные базы можно представить в виде взаимосвязанных таблиц.

Магазин

Товары

Название	Количество	Цена
Стол	2	3 000
Стул	5	1 000
Табурет	1	500

Клиенты

Имя	Телефон	Код
Миша	+7 920...	1
Саша	+7 930...	2
Наташа	+7 940...	3

Покупки

Название	Количество	Код
Табурет	1	3

Основные понятия

Деталь

Атрибут
(столбец, поле)

Отношение/Таблица

Кортеж
(строка, запись)

Номер	Название	Цвет	Вес	Номер скада
101	болт	черный	3	1
102	винт	красный	9	1
103	гайка	зеленый	4	2
104	болт	синий	11	1
105	гайка	желтый	7	2
106	болт	синий	11	2

Первичный ключ

Первичный ключ (primary key) – уникальное значение для каждой строки. Может быть простым или составным

Первичный ключ

Номер	Название	Цвет	Вес	Номер скада
101	болт	черный	3	1
102	винт	красный	9	1
103	гайка	зеленый	4	2
104	болт	синий	11	1
105	гайка	желтый	7	2
106	болт	синий	11	2

Внешний ключ

Внешний ключ — это ключ, используемый для связывания двух таблиц друг с другом.

Первичный ключ

Внешний ключ

Номер	Название	Цвет	Вес	Номер скада
101	болт	черный	3	1
102	винт	красный	9	1
103	гайка	зеленый	4	2
104	болт	синий	11	1
105	гайка	желтый	7	2
106	болт	синий	11	2

Первичный ключ

Номер	Адрес
1	ул. Годовикова, 9, стр. 10, Москва
2	ул. Воздвиженка, 3/5, строение 1, Москва

ВОПРОСЫ

Нормальные формы

Нормальные формы

Обеспечивают снижение избыточности и возможной противоречивости данных

1. 1NF (Первая нормальная форма)
2. 2NF
3. 3NF

4. BCNF (Нормальная форма Бойса-Кодда)
5. 4NF
6. 5NF
7. DKNF (Доменно-ключевая нормальная форма)
8. 6NF

Нормальные формы

Обеспечивают снижение избыточности и возможной противоречивости данных

1. **1NF (Первая нормальная форма)**
2. **2NF**
3. **3NF**

4. BCNF (Нормальная форма Бойса-Кодда)
5. 4NF
6. 5NF
7. DKNF (Доменно-ключевая нормальная форма)
8. 6NF

Первая нормальная форма

Все атрибуты являются простыми

Фирма	Модели
BMW	M5, X5M, M1
NISSAN	GT-R

Первая нормальная форма

Все атрибуты являются простыми

Фирма	Модели
BMW	M5, X5M, M1
NISSAN	GT-R

→

Фирма	Модели
BMW	M5
BMW	X5M
BMW	M1
NISSAN	GT-R

Вторая нормальная форма

Находится в 1НФ

Каждый не ключевой атрибут зависит от первичного ключа, а не его части

Модель	Фирма	Цена	Скидка
M5	BMW	5 500 000	5%
X5M	BMW	6 000 000	5%
M1	BMW	2 500 000	5%
GT-R	NISSAN	5 000 000	10%

Вторая нормальная форма

Находится в 1НФ

Каждый не ключевой атрибут зависит от первичного ключа, а не его части

Модель	Фирма	Цена	Скидка
M5	BMW	5 500 000	5%
X5M	BMW	6 000 000	5%
M1	BMW	2 500 000	5%
GT-R	NISSAN	5 000 000	10%

Модель	Фирма	Цена
M5	BMW	5 500 000
X5M	BMW	6 000 000
M1	BMW	2 500 000
GT-R	NISSAN	5 000 000

Фирма	Скидка
BMW	5%
NISSAN	10%

Третья нормальная форма

Находится во 2НФ
Каждый не ключевой атрибут должен зависеть только от первичного ключа

Фирма	Магазин	Телефон
BMW	Риал-авто	87-33-98
Audi	Риал-авто	87-33-98
Nissan	Некст-Авто	94-54-12

Третья нормальная форма

Находится во 2НФ
Каждый не ключевой атрибут должен зависеть только от первичного ключа

Фирма	Магазин	Телефон
BMW	Риал-авто	87-33-98
Audi	Риал-авто	87-33-98
Nissan	Некст-Авто	94-54-12

Фирма	Магазин
BMW	Риал-авто
Audi	Риал-авто
Nissan	Некст-Авто

Магазин	Телефон
Риал-авто	87-33-98
Некст-Авто	94-54-12

Нормальные формы

Нормализация БД

- Преобразования базы данных к нормальным формам
- Приведение структуры БД к виду, обеспечивающему минимальную логическую избыточность

Нормальные формы

Нормализация БД

- Преобразования базы данных к нормальным формам
- Приведение структуры БД к виду, обеспечивающему минимальную логическую избыточность
- Уменьшение потенциальной противоречивости информации

Нормальные формы

Нормализация БД

- Преобразования базы данных к нормальным формам
- Приведение структуры БД к виду, обеспечивающему минимальную логическую избыточность
- Уменьшение потенциальной противоречивости информации
- Не имеет целью увеличение производительности работы

ВОПРОСЫ

SQL

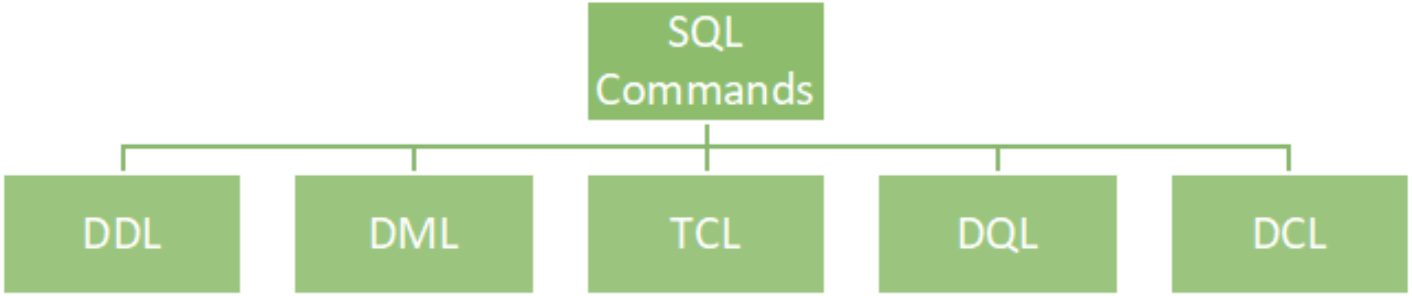
SQL

Structured Query Language - язык структурированных запросов

SQL
Commands

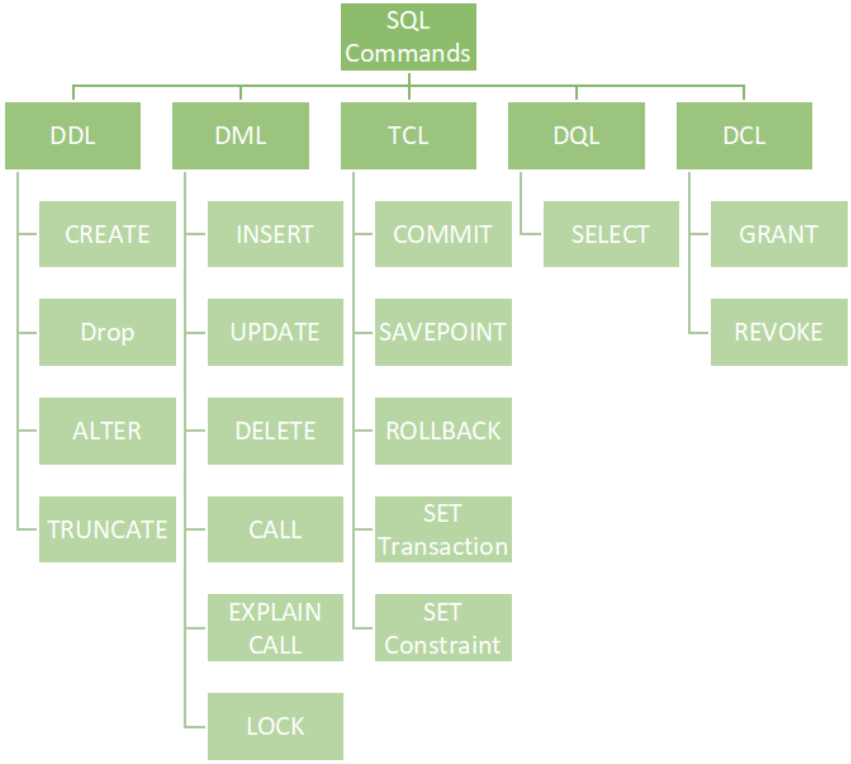
SQL

Structured Query Language - язык структурированных запросов



SQL

Structured Query Language - язык структурированных запросов



SQL

SQL. Типы данных

Имя	Псевдонимы	Описание
boolean	bool	логическое значение (true/false)
character [(n)]	char [(n)]c	символьная строка фиксированной длины
character varying [(n)]	varchar [(n)]	символьная строка переменной длины
date		календарная дата (год, месяц, день)
double precision	float8	число двойной точности с плавающей точкой (8 байт)
integer	int, int4	знаковое четырёхбайтное целое
json		текстовые данные JSON
jsonb		двоичные данные JSON, разобранные
real	float4	число одинарной точности с плавающей точкой (4 байта)
text		символьная строка переменной длины
timestamp		дата и время
uuid		универсальный уникальный идентификатор

Подробнее тут: https://postgrespro.ru/docs/postgresql/17/datatype

SQL. CREATE TABLE

```
CREATE TABLE имя_таблицы (
  { имя_столбца тип_данных [опции] },
  { имя_столбца тип_данных [опции] },
  …..
  { имя_столбца тип_данных [опции] },
 );
```

Подробнее тут: https://postgrespro.ru/docs/postgresql/17/sql-createtable

SQL. CREATE TABLE

Параметры столбцов

primary key - обозначить столбец как первичный ключ

not null - запретить запись null'а в колонку

default значение - значение по умолчанию (если значение не указано)

references имя_таблицы(имя_колонки) - указать внешний ключ

now() - получить значение текущего времени, обычно используется как **default now()**

…

SQL. CREATE TABLE

Пример

Требуется хранить информацию о клиентах компании – работодателях:
- название
- регион

SQL. CREATE TABLE

Пример

Требуется хранить информацию о клиентах компании – работодателях:
- название
- регион

```
CREATE TABLE employer (
    employer_id serial primary key,
    employer_name text not null,
    area_id integer not null
);
```

SQL. CREATE TABLE

Пример

Требуется хранить информацию о вакансиях клиентов:
- наименование позиции
- зарплата от
- зарплата до

SQL. CREATE TABLE

Пример

Требуется хранить информацию о вакансиях клиентов:
- наименование позиции
- зарплата от
- зарплата до

```
CREATE TABLE vacancy (
    vacancy_id serial primary key,
    employer_id integer not null references employer (employer_id),
    position_name text not null,
    compensation_from integer,
    compensation_to integer
);
```

SQL. CREATE TABLE

Пример

В вакансиях не хватает понимания о зарплате – до вычета налогов или после.

SQL. CREATE TABLE

Пример

В вакансиях не хватает понимания о зарплате – до вычета налогов или после.

```
ALTER TABLE vacancy ADD COLUMN compensation_gross boolean;
```

SQL. ALTER TABLE

```
ALTER TABLE имя_таблицы действие;

ALTER TABLE имя_таблицы ADD COLUMN описание;
ALTER TABLE имя_таблицы DROP имя_колнки;
ALTER TABLE имя_таблицы RENAME COLUMN  имя_столбца TO новое_имя_столбца;

ALTER TABLE vacancy ADD COLUMN compensation_gross boolean;
```

Подробнее тут: https://postgrespro.ru/docs/postgresql/17/sql-altertable

SQL. INSERT

```
INSERT INTO имя_таблицы (имя_столбца1, имя_столбца2, … имя_столбцаN)
VALUES
(значение_для_столбца1, значение_для_столбца2, ... значение_для_столбцаN),
(значение_для_столбца1, значение_для_столбца2, … значение_для_столбцаN),
…
(значение_для_столбца1, значение_для_столбца2, ... значение_для_столбцаN);
```

Подробнее тут: https://postgrespro.ru/docs/postgresql/17/sql-insert

SQL. INSERT

Пример

```
INSERT INTO employer (employer_name, area_id)
VALUES ('hh.ru', 1),
       ('Яндекс', 1),
       ('Рога и Копыта', 2),
       ('Google', 2),
       ('Иванов и Ко', 3);
```

SQL. INSERT

Пример

```
INSERT INTO employer (employer_name, area_id)
VALUES ('hh.ru', 1),
       ('Яндекс', 1),
       ('Рога и Копыта', 2),
       ('Google', 2),
       ('Иванов и Ко', 3);
```

```
INSERT INTO vacancy (employer_id, position_name,
compensation_from, compensation_to, compensation_gross)
VALUES (1, 'Разработчик', 20000, 30000, true),
       (2, 'Тестировщик', 10000, 20000, true),
       (2, 'Аналитик', 15000, 17000, false),
       (3, 'Бухгалтер', 20000, 30000, false);
```

SQL. SELECT

```
SELECT * | колонки | выражение
FROM название_таблицы
ORDER BY сортировка [ASC | DESC]
LIMIT n
OFFSET m;
```

Подробнее тут: https://postgrespro.ru/docs/postgresql/17/sql-select

SQL. WHERE

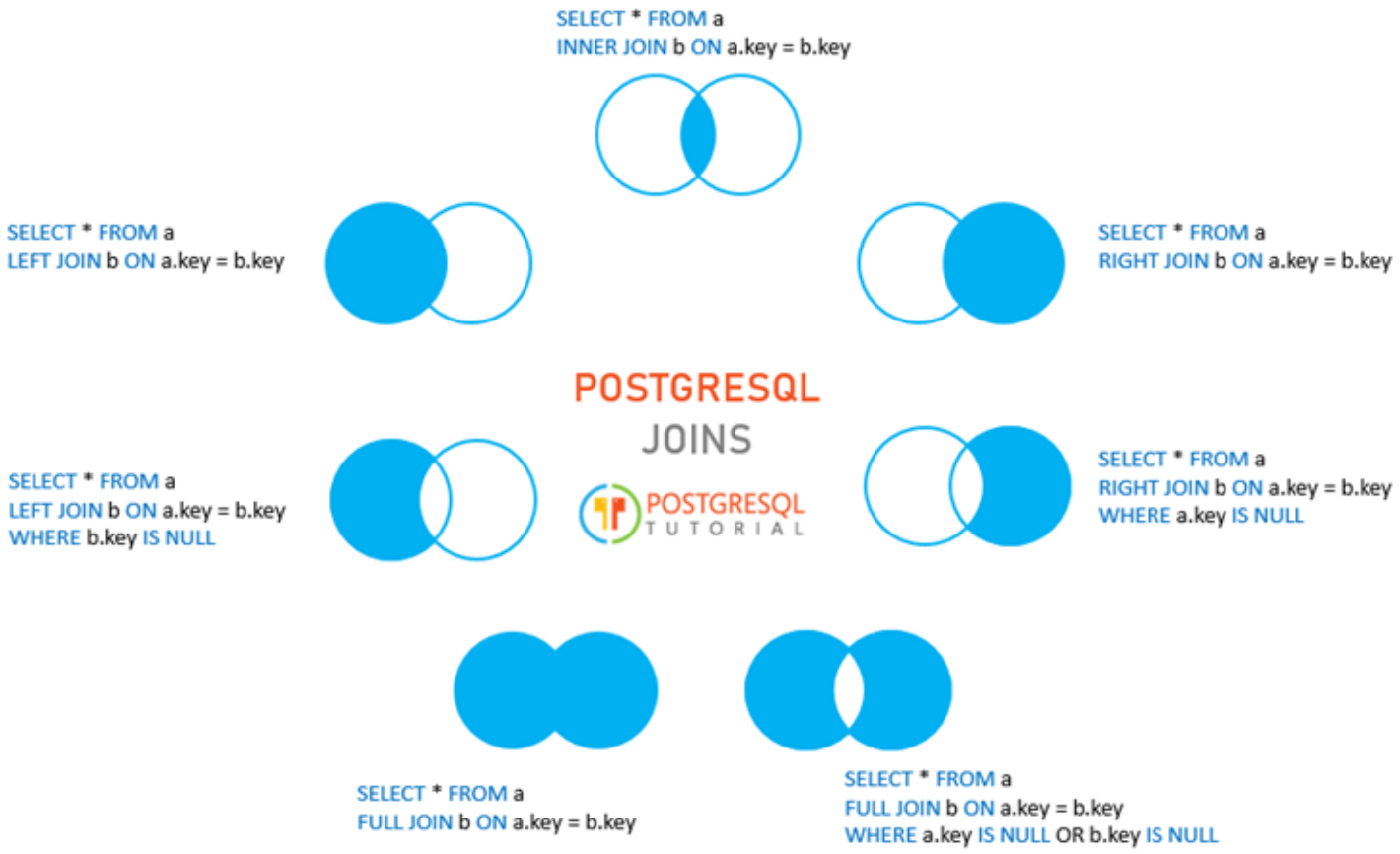
Примеры

- Операторы сравнения **<, <=, >, >=, =** или **IS, <>** (не равно)
- Между значениями **BETWEEN A AND B**
- Соответствие паттерну **LIKE '_some text%'**
  '**_**' соответствует любому символу
  '**%**' соответствует любой последовательности символов (в т.ч. пустой)
- Принимает одно из значений **IN (A, B, …., C)**

SQL. JOINS

Требуется получить информацию о всех работодателях и их вакансиях

SQL. JOINS


SELECT * FROM a
INNER JOIN b ON a.key = b.key
SELECT * FROM a
LEFT JOIN b ON a.key = b.key
SELECT * FROM a
RIGHT JOIN b ON a.key = b.key
POSTGRESQL
JOINS
POSTGRESQL
TUTORIAL
SELECT * FROM a
LEFT JOIN b ON a.key = b.key
WHERE b.key IS NULL
SELECT * FROM a
RIGHT JOIN b ON a.key = b.key
WHERE a.key IS NULL
SELECT * FROM a
FULL JOIN b ON a.key = b.key
SELECT * FROM a
FULL JOIN b ON a.key = b.key
WHERE a.key IS NULL OR b.key IS NULL

SQL. JOINS

```sql
SELECT employer_name, position_name
FROM employer e
LEFT JOIN vacancy v ON e.employer_id = v.employer_id;


SELECT employer_name, position_name
FROM employer e
INNER JOIN vacancy v ON e.employer_id = v.employer_id;
```

SQL. Агрегатные функции

Примеры

- **COUNT** – количество элементов
- **SUM** – сумма элементов
- **MIN** – минимум
- **MAX** – максимум
- **AVG** – среднее арифметическое

Подробнее тут: https://postgrespro.ru/docs/postgrespro/17/functions-aggregate

SQL. Агрегатные функции

```
SELECT min(compensation_from) AS min_salary,
       max(compensation_to) AS max_salary,
       avg(compensation_to - compensation_from) AS avg_salary_range,
       count(DISTINCT vacancy_id) AS vacancies_count
FROM vacancy;
```

SQL. GROUP BY

SQL. GROUP BY

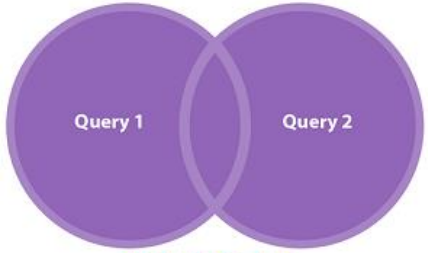
```sql
SELECT area_id,
       min(compensation_from)                    AS min_salary,
       max(compensation_to)                      AS max_salary,
       avg(compensation_to - compensation_from) AS avg_salary_range,
       count(DISTINCT vacancy_id)                AS vacancies_count
FROM employer e
         LEFT JOIN vacancy v on e.employer_id = v.employer_id
GROUP BY area_id
HAVING count(vacancy_id) > 0;
```

SQL. GROUP BY

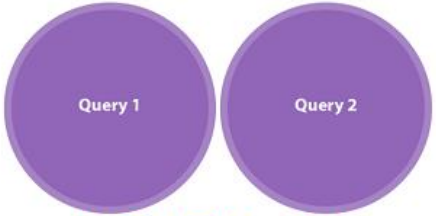
ВОПРОСЫ

SQL. Операции над множествами

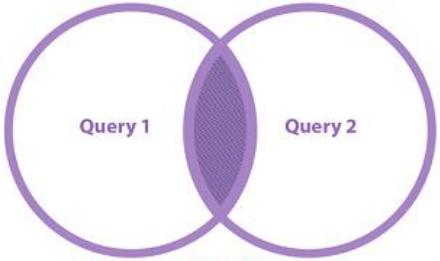
- UNION
- UNION ALL
- INTERSECT
- EXCECT



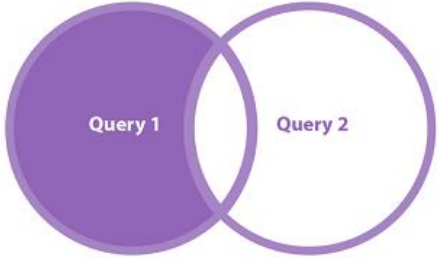
UNION



UNION ALL



INTERSECT



EXCEPT

SQL. Операции над множествами

```
SELECT employer_id AS id, employer_name AS title
FROM employer
UNION
SELECT vacancy_id, position_name
FROM vacancy;
```

SQL. Условные выражения

```
SELECT v.position_name,
       CASE
           WHEN compensation_gross IS TRUE
               THEN compensation_to * 0.87
           ELSE compensation_to
           END AS compensation_to,
       CASE
           WHEN compensation_gross IS TRUE
               THEN compensation_from * 0.87
           ELSE compensation_from
           END AS compensation_from
FROM vacancy v;
```

SQL. Вложенные запросы

```
SELECT area_id, position_name, compensation_from, compensation_to
FROM vacancy v
         INNER JOIN employer e ON v.employer_id = e.employer_id
WHERE compensation_from = (
    SELECT compensation_from
    FROM vacancy
    WHERE position_name = 'Бухгалтер'
);
```

Перерыв

Транзакции

Транзакции

Транзакция — набор последовательных операций, который представляет собой логическую единицу работы с данными

Может быть выполнена либо целиком, либо не выполнена вообще

Транзакции

ACID

Набор требований к транзакционной системе, обеспечивающих максимально надежную и предсказуемую работу. Не все базы данных полностью реализуют **ACID**.

A atomicity (атомарность)
C consistency (согласованность)
I isolation (изолированность)
D durability (долговечность)

Транзакции

Аномалии

«Грязное» чтение. Могут быть считаны данные, которые были изменены откатившейся транзакцией.

Неповторяемое чтение. Данные оказываются измененными при повторном чтении в рамках одной транзакции

Фантомное чтение. Выборки по одним и тем же критериям внутри одной транзакции содержат разное количество строк

Аномалия сериализации. Результат фиксации транзакций оказывается несогласованным

Транзакции

Аномалии транзакций

Уровень изоляции	«Грязное» чтение	Неповторяемое чтение	Фантомное чтение	Аномалия сериализации
Read uncommited	Допускается, но не в PG	Возможно	Возможно	Возможно
Read commited	Невозможно	Возможно	Возможно	Возможно
Repeatable read	Невозможно	Невозможно	Допускается, но не в PG	Возможно
Serializable	Невозможно	Невозможно	Невозможно	Невозможно

Demo. Транзакции

Индексы

Индексы

+ Индексы позволяют ускорить поиск данных

- Замедляется запись данных

- Занимают место на диске

```
CREATE INDEX compensation_from_index ON vacancy (compensation_from);
DROP INDEX compensation_from_index;
```

Индексы

Типы индексов в PostgreSQL

- B-дерево
- Хеш
- GiST
- SP-GiST
- GIN
- BRIN

Подробнее тут: https://postgrespro.ru/docs/postgrespro/17/indexes-types

Индексы

B-дерево

Поддерживаемые условия:

- Операторы сравнения
- Сочетание операторов сравнения (BETWEEN, IN)
- IS NULL, IS NOT NULL,
- LIKE, если шаблон определяется константой или привязан к началу строки.

Пример: value LIKE 'some%'

Домашнее задание

Домашнее задание

0. Пройти опрос.

1. Спроектировать базу данных hh (основные таблицы: вакансии, резюме, отклики, специализации). По необходимым столбцам ориентироваться на сайт hh.ru

2. Заполнить базу данных тестовыми данными (порядка 10к вакансий и 100к резюме)

3. Написать запрос для получения средних значений по регионам (area_id) следующих величин: compensation_from, compensation_to, среднее_арифметическое_from_и_to

4. Написать запрос для получения месяца с наибольшим количеством вакансий и месяца с наибольшим количеством резюме

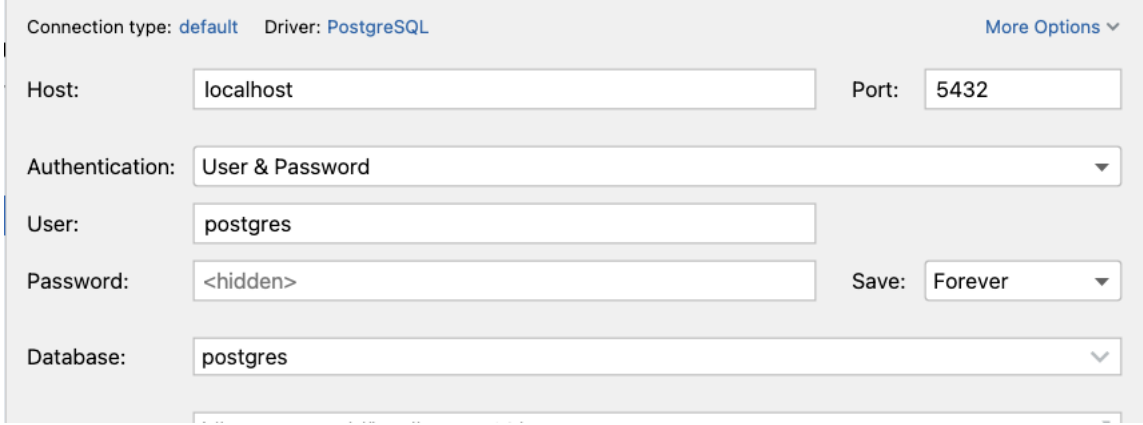
5. Написать запрос для получения id и title вакансий, которые собрали больше 5 откликов в первую неделю после публикации

6. Создать необходимые индексы (обосновать выбор столбцов)

По стилю кода можно ориентироваться на https://gist.github.com/fredbenenson/7bb92718e19138c20591

Запуск PostgreSQL

1. Установить Docker https://docs.docker.com/engine/install/
2. Запустить контейнер с PostgreSQL:
   docker run --name lection -p 5432:5432 -e POSTGRES_PASSWORD=123 -d postgres:15.4
   где lection – имя контейнера
   5432:5432 – порт, на котором контейнер будет доступен снаружи (с локальной машины)
   123 – пароль от учётной записи
3. Настроить подключение в DataGrip:



4. Вы великолепны!

Спасибо за внимание